

AI FOR SOFTWARE ENGINEERING

PAULINE MWENJE JOMO: WEEK 5 ASSIGNMENT:

Part 1: Problem Definition

1. Hypothetical AI problem

Building an AI system that predicts, at ED triage, which patients are at high risk of needing ICU admission within the next 24–48 hours.

Objectives

1. Prioritize high-risk patients for closer monitoring, tests, and early interventions.
2. Improve ICU bed planning and resource allocation in the hospital.
3. Reduce preventable deterioration and mortality by identifying critical patients earlier.

Two stakeholders

1. Emergency Department clinicians (doctors, nurses, triage staff) who make real-time decisions.
2. Hospital administrators and ICU managers who plan staffing and bed capacity.

Key Performance Indicator: Reduction in unplanned ICU transfers from general wards (i.e., more patients get to ICU early instead of deteriorating on the ward).

2. Data Collection & Preprocessing

Two data sources

1. **ED triage and EHR data:** vital signs at arrival (heart rate, blood pressure, oxygen saturation, respiratory rate, temperature), chief complaint, triage category, comorbidities, medications.
2. **Laboratory and imaging results (early tests):** key lab values ordered in the ED (e.g., lactate, white blood cell count), and high-level imaging findings where available.

Potential bias

If historical data comes mainly from certain times (e.g., day shifts) or specific subpopulations (e.g., patients who tend to come to this hospital), the model may perform worse for underrepresented groups such as patients arriving at night, from rural areas, or with atypical presentations, leading to unequal care.

Three preprocessing steps

1. Data cleaning and missing values:

- Remove duplicates and fix inconsistent IDs.
- Impute missing vitals or lab values where clinically appropriate, or add “missing” indicators.

2. Feature engineering:

- Calculate early warning scores or aggregate vitals into derived features (e.g., shock index = heart rate / systolic BP).
- Add flags for key comorbidities (e.g., chronic heart failure, COPD).

3. Encoding and splitting:

- Encode categorical variables (chief complaint category, triage level).
- Scale numeric features where needed.
- Split into training, validation, and test sets, ideally using a time-based split.

3. Model Development

Model choice and justification

I would choose a **Gradient Boosted Trees model**. This model handles heterogeneous tabular clinical data well, captures nonlinear relationships between vitals and lab results, can cope with missing values, and provides feature importance, which will help clinicians understand which factors drove a high-risk prediction. A simpler baseline (e.g., logistic regression) could be used for comparison.

How to split data into training/validation/test sets

Use a temporal split to avoid “peeking into the future”:

- **Training set:** older ED visits (e.g., data from earlier months/years).

- **Validation set:** more recent visits, used for hyperparameter tuning and model selection.
- **Test set:** the most recent period, held out until the very end to estimate future performance.

Two hyperparameters and why

1. **Max depth:** controls how deep each tree can grow. Limiting depth helps prevent overfitting to noise in clinical data and improves generalization.
2. **Learning rate (for boosting) or n_estimators (number of trees):** controls how many trees are combined and how strongly each update changes the model. Tuning this balances performance against overfitting and training time.

4. Evaluation & Deployment.

Two evaluation metrics and their relevance

1. **Recall (Sensitivity) for the “ICU admission” class:**
 - Important because missing a patient who truly needs ICU (false negative) can lead to serious harm.
2. **Precision or F1-score for the “ICU admission” class:**
 - High precision avoids overwhelming ICU resources or staff with too many false alarms.
 - F1-score balances precision and recall, helpful when ICU cases are relatively rare.

What is concept drift? How would you monitor it post-deployment?

Concept drift is when the relationship between input features and ICU admission risk changes over time. For example, new treatment protocols, different triage guidelines, seasonal disease patterns, or a pandemic can change which patients are likely to need ICU.

To monitor drift after deployment, I would:

- Track performance metrics (recall, precision, F1-score, ROC-AUC) regularly (e.g., monthly or quarterly).
- Compare recent feature distributions (vitals, lab results, triage categories) against historical distributions.
- Periodically retrain or update the model when performance drops or input patterns shift significantly.

One technical challenge during deployment

A key challenge is **real-time integration and latency**. The model must:

- Ingest triage data and early lab results quickly.
- Return a risk score within seconds so it fits into the ED's high-pressure workflow.

This requires efficient infrastructure, robust APIs connected to the EHR, and careful design to avoid slowing down triage or overwhelming staff with alerts.

Part 2: Case Study Application

Scenario

A hospital wants an AI system to predict patient readmission risk within 30 days of discharge.

1. Problem Scope

Develop a predictive model that estimates the probability that a patient will be readmitted within 30 days of discharge. The model should support clinicians in planning follow-up care and resource allocation.

Objectives

1. Identify high-risk patients at discharge so clinicians can arrange appropriate follow-up, education, or home care.
2. Reduce avoidable readmissions, improving patient outcomes and satisfaction.
3. Support hospital management in meeting regulatory and reimbursement targets related to readmission rates.

Stakeholders

- Primary: Patients, clinicians (doctors, nurses, care coordinators).
- Secondary: Hospital administrators, insurers, and regulators.

2. Data Strategy

Proposed data sources

1. Electronic Health Records (EHRs): diagnoses, procedures, comorbidities, lab tests, medications, vital signs, length of stay, previous admissions.
2. Demographic and social data: age, gender, socio-economic indicators, living situation, distance from the hospital, insurance type.

Two ethical concerns

1. Patient privacy and confidentiality: health data is highly sensitive, so strict access control, encryption, and compliance with regulations such as HIPAA are essential.
2. Bias and fairness: if certain groups (for example, older adults, people from certain neighborhoods) are overrepresented in readmissions, the model might reinforce inequities by labeling them as “high risk” without addressing underlying social determinants of health.

Preprocessing pipeline and feature engineering

1. Data cleaning: remove duplicate records, resolve conflicting entries, and handle missing lab values (for example, impute or flag missingness as a feature).
 2. Feature engineering:
 - Aggregate prior admissions in the past year.
 - Compute comorbidity indices (for example, Charlson-like scores) from diagnosis codes.
 - Derive features such as length of stay, ICU admission flag, or discharge destination (home, rehab facility).
 3. Encoding and scaling:
 - Encode categorical variables (diagnosis groups, discharge destination, insurance type).
 - Normalize or standardize continuous variables (age, lab results).
 4. Train–test split: create a temporally aware split so that training uses older data and testing uses more recent discharges, better reflecting real deployment conditions.
-

3. Model Development

Model selection and justification

I would start with a Gradient Boosted Trees model (for example XGBoost or similar) or a Random Forest classifier. These models handle nonlinear relationships, work well with tabular medical data, can manage missing values reasonably, and provide feature importance, which helps with interpretability in healthcare.

Confusion matrix with hypothetical data

Assume on a test set of 1,000 patients:

- 200 are truly readmitted within 30 days (positives).
- 800 are not readmitted (negatives).

Hypothetical model predictions:

- True Positives (TP): 140
- False Positives (FP): 60
- False Negatives (FN): 60
- True Negatives (TN): 740

Confusion matrix:

	Predicted Readmit	Predicted No Readmit
Actual Readmit	TP = 140	FN = 60
Actual No Readmit	FP = 60	TN = 740

Precision and recall

- Precision = $TP / (TP + FP) = 140 / (140 + 60) = 140 / 200 = 0.70$
- Recall = $TP / (TP + FN) = 140 / (140 + 60) = 140 / 200 = 0.70$

So the model correctly identifies 70 percent of actual readmissions, and 70 percent of predicted readmissions are correct.

4. Deployment

Integration steps

1. API or service layer: deploy the model as a REST API or internal service that can receive patient data at discharge and return a readmission risk score.
2. EHR integration: integrate with the hospital's EHR so that when a discharge order is entered, relevant features are automatically extracted and sent to the model.
3. User interface: display the risk score and key contributing features in a clinician-friendly dashboard within the EHR. For example, highlight top risk factors such as comorbidities or frequent past admissions.
4. Workflow design: define how clinicians should respond to high-risk scores (for example, schedule follow-up appointments, arrange social work consults, provide discharge education).

5. Logging and monitoring: log model inputs, outputs, and subsequent outcomes to monitor performance and support auditability.

Ensuring regulatory compliance

- Data protection: apply strong encryption, role-based access control, and secure data transmission in line with HIPAA requirements (or equivalent).
 - Audit trails: maintain detailed logs of model usage for compliance and investigation of any incidents.
 - Governance: involve legal, compliance, and clinical governance teams in model approval and updates.
 - Transparency: document data sources, model behavior, and limitations so clinicians understand how to appropriately use the risk scores.
-

5. Optimization

One method to address overfitting

- Use cross-validation with regularization. For example, in a gradient boosting model, limit the tree depth, reduce the learning rate, and apply early stopping based on validation performance. This prevents the model from memorizing training data and improves generalization to new patients.

Part 4: Reflection and Workflow Diagram

Reflection

Most challenging part and why

The most challenging part of the workflow was defining the problem and data strategy in a way that is both clinically meaningful and technically feasible. It required balancing what data is realistically available, what is ethically acceptable to use, and what will actually influence clinical decisions.

How I would improve my approach

With more time and resources, I would:

- Involve clinicians, data engineers, and compliance officers earlier in the design phase to refine objectives and constraints.
- Conduct a more thorough error analysis of the model, exploring which types of patients are misclassified and why.

- Experiment with multiple models and interpretability tools (for example SHAP values) to provide richer, more actionable explanations for clinicians.

2. AI Development workflow Diagram

